WORKSHEET 4

Student Name: Analava Bera **UID: 23BCS13611**
Branch: BE-CSE **Section/Group: Krg_3A**
Semester: 6th **Subject Name: System Design**
Subject Code: 23CSH-314

Aim: To design a video streaming application
Steps:

1. **Functional Requirements:**
   1.Client should be able to create account on the OTT platform.
   2. After the successfull login, client should be able to opt for the subscription plans.
   3. Client should be able to search for the shows/movies based on the video title or names.
   4. Client should be able to watch the videos / tv shows in multiple different resolutions (480p,720p,1080p, 4k etc.)
   5. Recommendation for TV shows and movies.

2. **Non-Functional Requirements:**
   1. **Scalability:** 200-300M, for which let's say total videos we are having are 20K videos (~1 hour each)
   2. **CAP Theorem**: Availability >>>>> Consistency
      Availability on watching TV shows and movies
      Consistency in making payments and in subscription plans
   3. **Latency:** 50 - 80 ms.
      Client should be able to see the video with zero or neglibile buffering.

3. **API Design:**
   A. User On-boarding API's
   1. **POST Call:** **https://www.netlfix.com/user/register**
   2. **POST Call:** https://www.netlfix.com/user/login
   3. **PUT Call**: https://www.netlfix.com/user/update

Subscription:

1. GET Call: https://www.netlfix.com/search?q={movie_name}
   Response: List<Video_ID> + some meta data of video -> Pagination

2. GET Call: https://www.netlfix.com/{video_ID}
   Response: Metadata of the video (JSON)

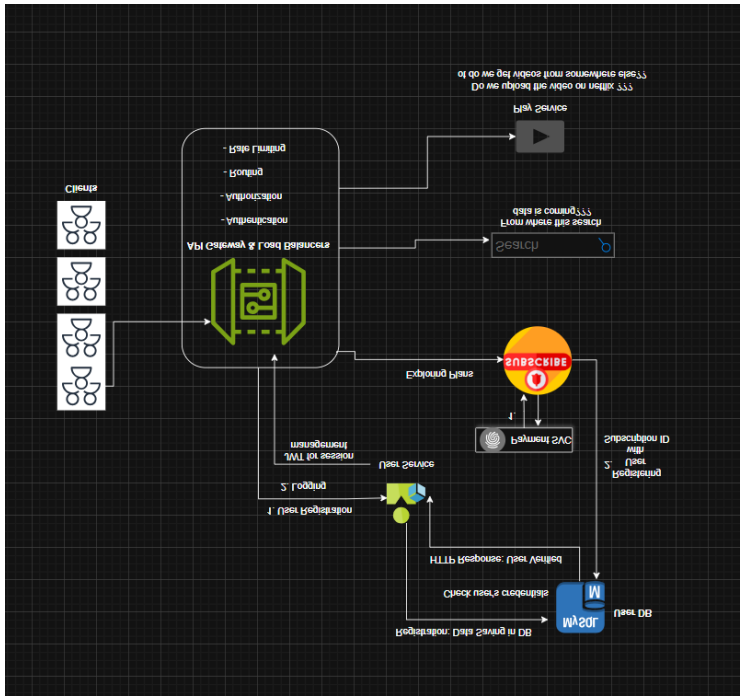
3.GET Call: https://www.netlfix.com/play/{video_ID}

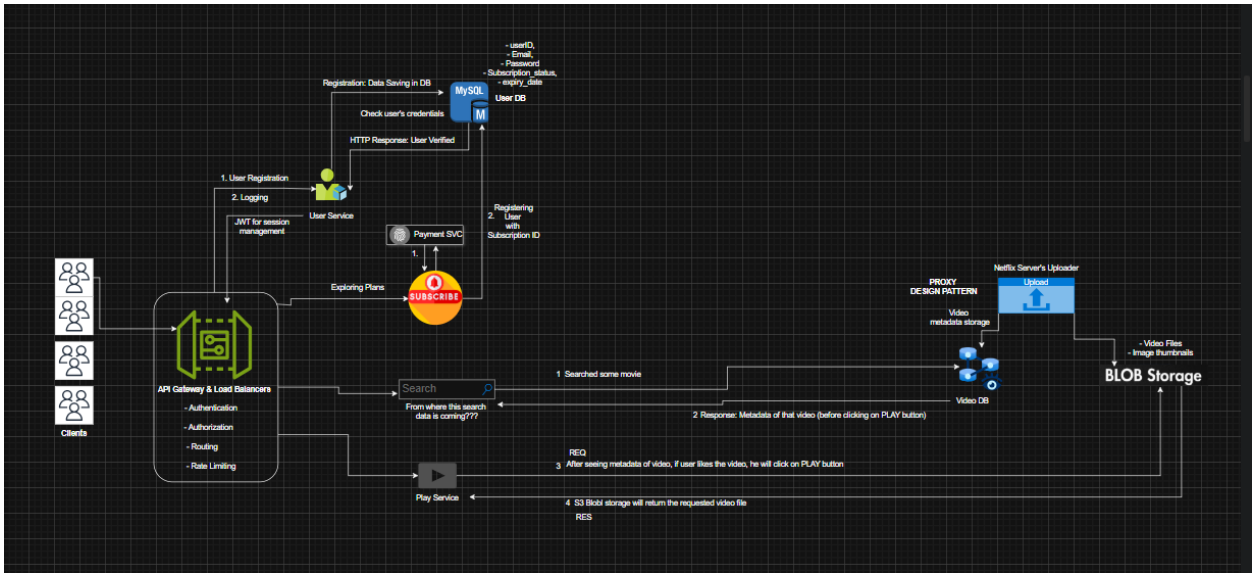
4. High-Level Design:

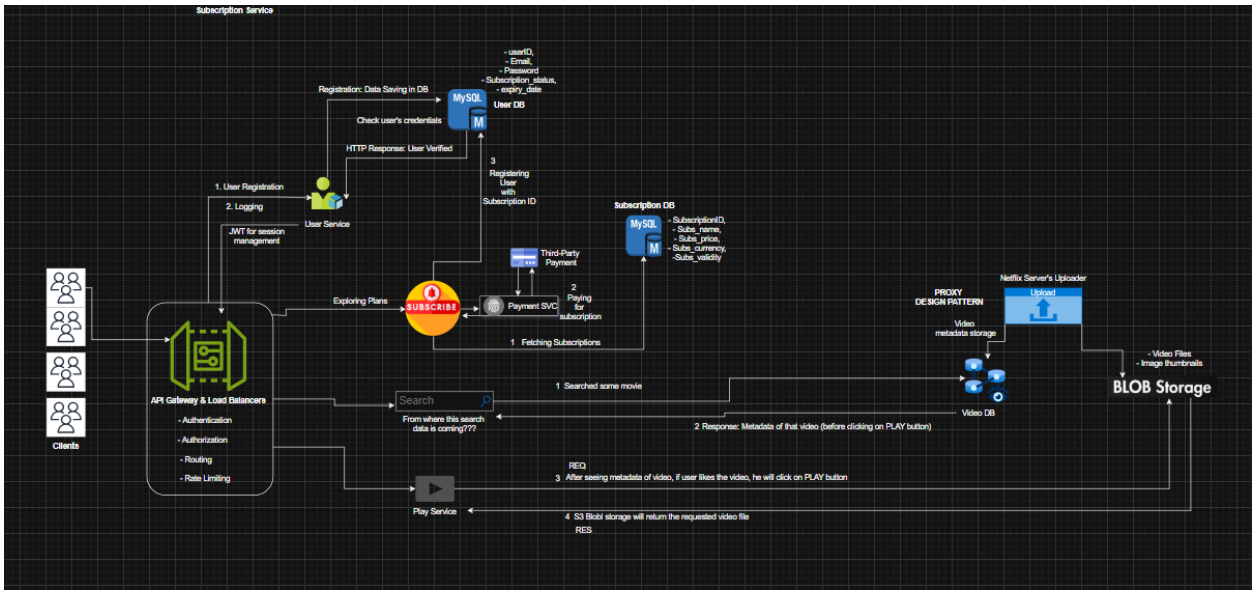
Now According to the functional requirement of the system, we can identify that :

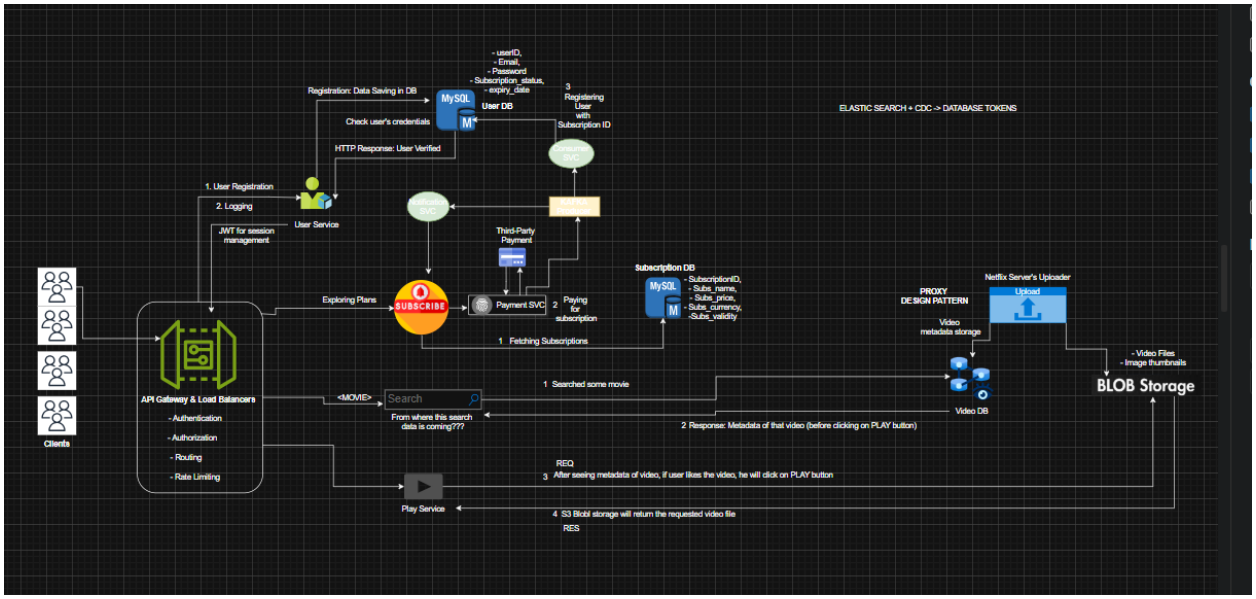
There will be a client who is requesting, then there will a server upon which computation will be going on, and lastly there will be an database in which storage will be done.

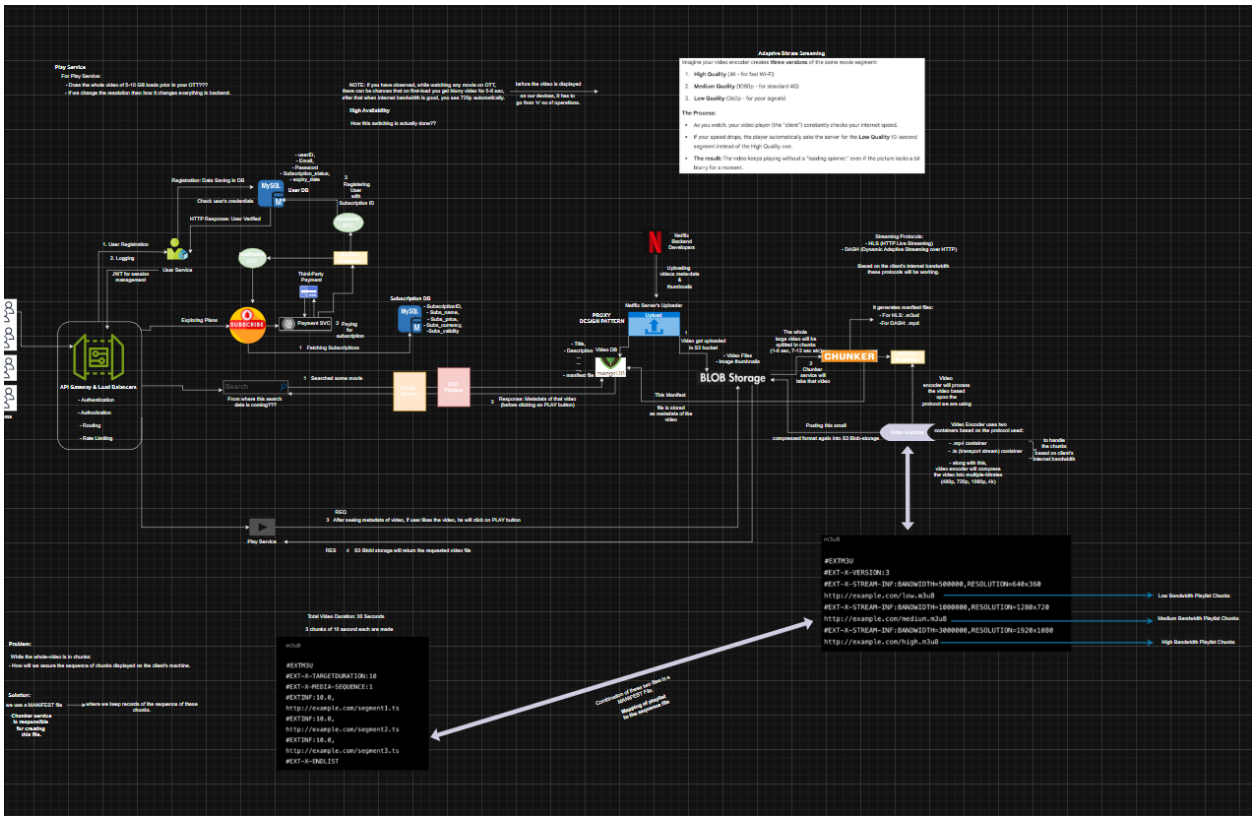
5. Low-Level Design:

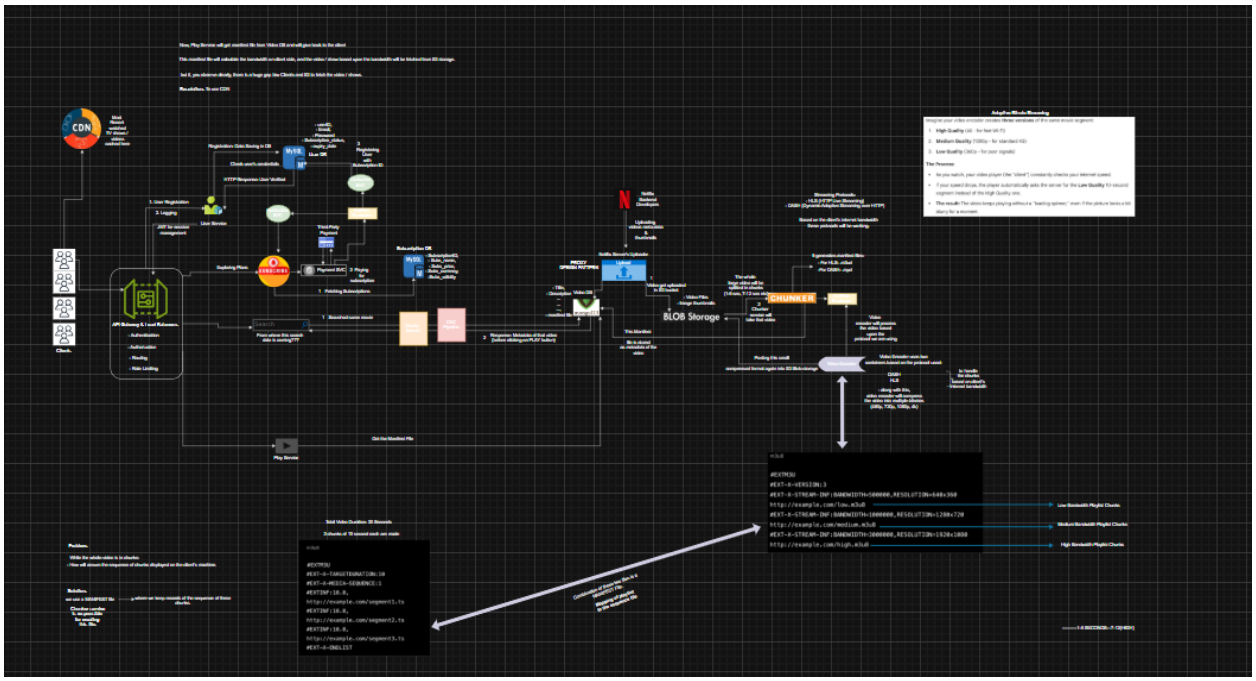



- userID,
- Email,
- Password
- Subscription_status,
- expiry_date
Registration: Data Saving in DB
MySQL
User DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Registering User with Subscription ID
Payment SVC
1.
Exploring Plans
SUBSCRIBE
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
From where this search data is coming???
1 Searched some movie
2 Response: Metadata of that video (before clicking on PLAY button)
Netflix Server's Uploader
Upload
PROXY DESIGN PATTERN
Video metadata storage
Video DB
- Video Files
- Image thumbnails
BLOB Storage
REQ
3 After seeing metadata of video, if user likes the video, he will click on PLAY button
Play Service
4 S3 Blobl storage will return the requested video file
RES


Subscription Service
- userID,
- Email,
- Password
- Subscription_status,
- expiry_date
Registration: Data Saving in DB
MySQL
User DB
Check user's credentials
HTTP Response: User Verified
3
Registering User with Subscription ID
1. User Registration
2. Logging
User Service
JWT for session management
Subscription DB
MySQL
- SubscriptionID,
- Subs_name,
- Subs_price,
- Subs_currency,
-Subs_validity
Third-Party Payment
Exploring Plans
SUBSCRIBE
Payment SVC
2
Paying for subscription
1 Fetching Subscriptions
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
From where this search data is coming???
1 Searched some movie
2 Response: Metadata of that video (before clicking on PLAY button)
Netflix Server's Uploader
Upload
PROXY DESIGN PATTERN
Video metadata storage
Video DB
- Video Files
- Image thumbnails
BLOB Storage
REQ
3 After seeing metadata of video, if user likes the video, he will click on PLAY button
Play Service
4 S3 Blobl storage will return the requested video file
RES


- userID,
- Email,
- Password
- Subscription_status,
- expiry_date
Registration: Data Saving in DB
MySQL
User DB
3 Registering User with Subscription ID
Check user's credentials
HTTP Response: User Verified
ELASTIC SEARCH + CDC -> DATABASE TOKENS
1. User Registration
2. Logging
User Service
JWT for session management
Third-Party Payment
Subscription DB
- SubscriptionID,
- Subs_name,
- Subs_price,
- Subs_currency,
-Subs_validity
Netflix Server's Uploader
Upload
PROXY DESIGN PATTERN
Video metadata storage
Exploring Plans
SUBSCRIBE
Payment SVC
2 Paying for subscription
1 Fetching Subscriptions
- Video Files
- Image thumbnails
BLOB Storage
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
<MOVIE>
Search
1 Searched some movie
From where this search data is coming???
Video DB
2 Response: Metadata of that video (before clicking on PLAY button)
REQ
3 After seeing metadata of video, if user likes the video, he will click on PLAY button
Play Service
4 S3 Blobl storage will return the requested video file
RES


Play Service
For Play Service:
- Does the whole video of 5-10 GB loads prior in your OTT???
- If we change the resolution then how it changes everything in backend.
NOTE: If you have observed, while watching any movie on OTT, there can be chances that on first-load you get blurry video for 5-6 sec, after that when internet bandwidth is good, you see 720p automatically.
High Availability
How this switching is actually done??
before the video is displayed
on our devices, it has to go from 'n' no of operations.
Adaptive Bitrate Streaming
Imagine your video encoder creates three versions of the same movie segment:
1. High Quality (4k - for fast Wi-Fi)
2. Medium Quality (1080p - for standard 4G)
3. Low Quality (360p - for poor signals)
The Process:
As you watch, your video player (the "client") constantly checks your internet speed.
If your speed drops, the player automatically asks the server for the Low Quality 10-second segment instead of the High Quality one.
The result: The video keeps playing without a "loading spinner," even if the picture looks a bit blurry for a moment.
Netflix Backend Developers
Uploading videos metadata & thumbnails
Streaming Protocols:
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)
Based on the client's internet bandwidth these protocols will be working.
Video gets uploaded in S3 bucket
The whole large video will be splitted in chunks (1-4 sec, 1-12 sec etc)
CHUNKER
Chunker service will take that video
It generates manifest files:
-For HLS: .m3u8
-For DASH: .mpd
Video encoder will process the video based upon the protocol we are using.
Video Encoder uses two containers based on the protocol used:
- .mpd container
- .ts (transport stream) container
- along with this, video encoder will compress the video into multiple choices (480p, 720p, 1080p, 4k)
In handle the chunks based on client's internet bandwidth
Pushing this small compressed format again into S3 Blob storage
This identifies
file is stored as metadata of the video
Total Video Duration: 30 Seconds
3 chunks of 10 second each are made
m3u8
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
m3u8
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Problem:
While the whole-video is in chunks:
- How will we ensure the sequence of chunks displayed on the client's machine.
Solution:
we use a MANIFEST file
Chunker service is responsible for creating this file.
where we keep records of the sequence of these chunks.
Continuously it keeps hit/ fires a MANIFEST file
Showing all chunks in the sequence file


CDN
Clients
API Gateway & Load Balancer
Authentication
Authorization
Routing
Rate Limiting
Play Service
Check the Manifest File
Payment Gateway
Subscription Plans
Check user's credentials
HTTP Response (And Verified)
Registration Data Storing in DB
Third Party Payment Gateway
Payment GUI
Redis
Redis stores the user's current watch state
Response: Metadata of that video (Select stuff to get PLAY button)
Netflix Content Delivery Network
Uploading video to Netflix Backends
Netflix Encoder/Optimizer
Video get uploaded in S3 bucket
Video File Image Thumbnails
BLOB Storage
The Manifest file is stored as metadata of the video
CHUNKER
Chunker receives and take that video
Video encoder will process the video based upon the preload set encoding
Video Encoder uses two resolutions based on the preload watch
Pushing the result compressed format again into S3 Bucket storage
CDN HLS
along with this, video encoder will compress the video into multiple resolutions (480p, 720p, 1080p, 4k)
In handle the clients based on client's internet bandwidth
Adaptive Bitrate Streaming
Streaming Protocols
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)
Based on the client's internet bandwidth these protocols will be working
Segmentation and Encoding
- For H.264 codec
- For DASH - mp4
The video is chunked into multiple segments (10 sec, 5-7 sec etc.)
Video Encoder will encode three versions of the same exact segment
1. High-Quality (4K - for fast Wi-Fi)
2. Medium-Quality (1080p - for standard HD)
3. Low-Quality (240p - for poor signals)
The Process:
- As you watch, your video player (the "client") constantly checks your internet speed.
- If your speed drops, the player automatically asks the server for the Low Quality TS second segment instead of the high Quality one.
- The result: The video keeps playing without a "loading spinner," even if the picture looks a bit blurry for a moment.
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Total Video Duration: 20 Seconds
Volume of 10 second each set made
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
Problem:
While the whole video is in chunks
- How will stream the sequence of chunks displayed on the client's machine.
Solution:
we use a M3U8PLAYLIST file
Chunker creates
1. sequence file
for ensuring
file flow.
where we keep records of the sequence of these chunks.
Connection of these two files is a
Manifest file
ensuring the sequence
of the chunks is in
the correct way
1.0 SECONDS - 7/10 (90%)